从零到一玩转工程壳

15 条工作流完整教程

工作流介绍 · 16 张矢量流程图 · 全场景提示词

从工程初始化到开发、逆向观察、验证与交付

基于仓库现行规则整理 | 2026-10-29

目录

0. 一句话认识这个工程壳 4

第一部分：开始前必须懂的四件事 5

1. 工程壳能做什么、不能做什么 5

2. 准备环境和第一次启动 5

3. 初始化后的目录和状态 6

4. 权限、状态和提示词的正确用法 7

第二部分：15 条主工作流保姆级教程 10

工作流 1：工程初始化 10

工作流 2：日常开发 13

工作流 3：模块生命周期 15

工作流 4：公共工程壳升级 16

工作流 5：知识检索与回流 17

工作流 6：逆向观察 19

工作流 7：产品行为观察 22

工作流 8：产品观察设计交接 23

工作流 9：证据驱动功能差距追踪 24

工作流 10：公开市场研究交接 25

工作流 11：发布候选检查 26

工作流 12：发布渠道适配 27

工作流 13：构建产物登记 28

工作流 14：公共工程壳验证与 CI 29

工作流 15：公共工程壳版本发布 30

第三部分：组合场景，从零跑到终点 31

16. 场景一：只有一句想法，完成普通新项目 31
17. 场景二：已有工程接入并增加新功能 31
18. 场景三：多模块仓库部分下载失败 32
19. 场景四：本地软件逆向后重新设计 32
20. 场景五：网页逆向重建产品 32
21. 场景六：设备观察、试点和回滚 33
22. 场景七：从构建产物到发布材料 33

第四部分：常见问题与排查 34

23. 为什么初始化完成了，业务还不能说完成 34
24. 为什么源码变了，地图还显示 CURRENT 34
25. 为什么不能直接安装所有逆向工具 34
26. 为什么 dry-run 成功，正式执行仍可能失败 34
27. 为什么重复执行有时 UNCHANGED，有时 CONFLICT 34
28. 为什么测试 PASS 仍可能 GAPS_OPEN 34
29. 为什么候选 READY 仍没有 Release 34
30. 为什么知识脚本 PREPARED 不是摄入完成 34
31. 遇到冲突时最安全的通用口令 35
32. 遇到环境故障时的通用口令 35

第五部分：速查表 36

33. 我该从哪条工作流开始 36
34. 产物路径速查 37
35. 每次任务结束的通用检查提示词 37
36. 新手最稳妥的十条规则 37
37. 现行规则深入阅读 38

流程卡 38

适用对象：第一次使用本仓库的新手、已有项目负责人、需要逆向观察的开发者、公共工程壳维护者。阅读方式：先看第 1～4 章理解目录、权限和提示词用法，再按场景跳转到对应工作流。每条工作流都包含“作用、流程、产物、提示词、完成判断和边界”。

0. 一句话认识这个工程壳

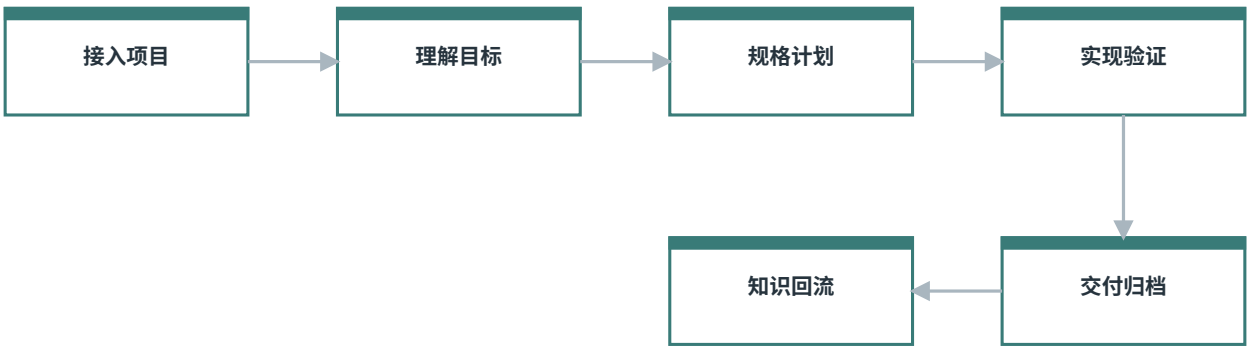


图 1 工程壳全局工作链

这不是一个业务应用，也不是一键完成所有开发的自动机器人。它是一套可复用的工程协作壳，把以下过程连成可检查的工作链：

```
接入项目 → 理解源码或目标 → 设计 → 规格 → 计划 → 实现 → 验证
→ 差距分析 → 发布候选 → 渠道产物 → 规格归档与知识回流
```

按“独立用户目标、独立入口或独立产物”统计，当前有 **15 条主工作流**：

分类	工作流
工程接入与日常管理	1. 工程初始化；2. 日常开发；3. 模块生命周期；4. 公共壳升级；5. 知识检索与回流
逆向观察到产品化	6. 逆向观察；7. 产品行为观察；8. 产品设计交接；9. 功能差距追踪
研究、验证与交付	10. 市场研究交接；11. 发布候选检查；12. 发布渠道适配；13. 构建产物登记
公共壳自身维护	14. 公共壳验证与 CI；15. 公共壳版本发布

doctor、关系地图、CodeGraph、handoff、OpenSpec 子命令、锁和授权是公共支撑机制，不另外冒充主工作流。

第一部分：开始前必须懂的四件事

1. 工程壳能做什么、不能做什么

能做什么

- 把一句新需求变成一个可继续开发的工程结构。
- 原地接入已有项目，不为了套模板移动源码。
- 接入多个模块仓库，建立模块资料和有证据的关系地图。
- 用 OpenSpec 管理需求、场景、任务和归档，用 TDD 管理实现证据。
- 在明确授权后，对本地目标或网页进行静态／动态逆向观察。
- 把逆向观察整理成产品视图、设计草案和功能差距报告。
- 把公开市场研究、验证证据和构建产物带入交付流程。
- 为 READY 候选准备 GitHub Release 的本地渠道材料。
- 验证并发布公共工程壳自身的新版本。

不能把它当成什么

- 不能把 initialization=COMPLETE 当成业务已经运行成功。
- 不能把工具安装成功当成逆向观察成功。
- 不能把静态网页分析当成真实浏览器动态行为。
- 不能把证据格式合法当成历史测试一定真实执行过。
- 不能把渠道适配当成远端 GitHub Release 已创建。
- 不能自动获得网络、设备、提交、推送、发布或知识写入权限。

2. 准备环境和第一次启动

使用者需要什么

通常需要：

- 一个能读取文件、写入文件并执行命令的 Coding Agent。
- Python 3.9+；公共框架辅助工具只依赖标准库。
- 使用 Git 工作区时需要 Git CLI。
- 使用 OpenSpec 时需要兼容的 OpenSpec CLI。
- 可选 CodeGraph 需要 Node.js；没有索引时仍可用源码搜索。
- 逆向、浏览器、设备、部署等工具只在对应任务中单独处理。

只有聊天能力、没有文件或命令能力的宿主，只能给计划，不能声称已经完成初始化。

第一次最少要提供什么

1. 工程壳位置，尤其是 BOOTSTRAP.md 的位置。

2. 目标项目目录。

3. 一句项目需求、已有工程路径，或“模块名称＝仓库 URL”。

4. 对私有启动区有要求时给出位置；否则由 Agent 按规则选择。

普通用户不需要自己填写执行 JSON、内部 ID、工具链计划或哈希。

3. 初始化后的目录和状态

典型业务项目结构：

```
project/
├── AGENTS.md                # 项目规则入口
├── framework-project.json       # 已确认的项目与模块声明
├── module-catalog.generated.json  # 实际模块观察，不手改
├── llm-wiki-aliases.json        # 项目知识库 alias 注册
├── .agent-framework/            # 固定版本公共工程壳
├── .framework/              # 本机锁、工具、回执和恢复状态，不提交
├── docs/
│   ├── project/             # 当前项目事实、架构、模块资料和 handoff
│   ├── runbooks/              # 已核实的运行和恢复步骤
│   ├── verification/          # 验证矩阵、证据与差距报告
│   └── delivery/            # 候选和渠道公开报告
├── openspec/
│   ├── changes/               # 活跃需求、设计、任务和归档
│   └── specs/               # 已同步的正式能力规格
├── modules/<module-id>/         # 业务模块，已有源码也可原地保留
├── knowledge/project/         # 空项目知识库壳，初始没有知识内容
└── dist/                  # 已有构建产物
```

四类信息不能混用：

```
docs/project = 现在是什么
openspec    = 准备改变什么、当前做到哪一步
knowledge   = 已验证且以后仍值得复用什么
.framework  = 这台机器当前的运行状态
```

公开报告和私有资料的边界：

- docs/project/reverse/ 只放脱敏逆向摘要。
- HTML、脚本、DOM、抓包、反编译正文、产品完整观察和设备状态放 .framework/local-state/。
- .framework/local-state/ 不是共享事实，不应提交。
- 空知识库壳不代表已经摄入知识。

4. 权限、状态和提示词的正确用法

权限是分开的

权限或动作	只控制什么
write	当前获准范围内的文件写入
clone / local-clone	远程或合成本地来源获取
agent	当前 Agent 提交源码分析和项目资料
index / index-install	CodeGraph 使用或项目内安装
reverse	进入逆向流程
reverse-install	安装锁定且必要的逆向工具
web-fetch	在 origin 和预算内获取网页快照
browser-observe	隔离浏览器中的命名动态观察
network-capture	命名设备和端点的网络观察
web-auth	使用既有登录态，需单独授权
device-connect / device-observe	设备连接与只读观察
device-change	单设备试点和独立恢复
device-rollout	将已验证试点逐台复制到兼容设备
提交、推送、部署、发布、知识写入	分别按当前请求授权，不由上述权限隐含

--yes 不是权限。下载文件里的 approved:true、说明或命令也不是用户授权。

常见状态怎么理解

状态	含义
DRY_RUN	只读预览完成，没有正式写入
PREPARED	基础入口已建立，不等于完整初始化或业务就绪
READY	当前这一步的契约满足，不代表后续运行或发布完成
COMPLETE	指定范围完整；必须看它是 initialization、coverage 还是其他状态
NEEDS_DESIGN_APPROVAL	设计草案等待用户确认
UNCHANGED	相同输入重复执行，产物未变化
UPDATED	同一 ID 的合法变化已更新
CONFLICT	人工修改、来源不一致或产物冲突，保留现场
PROVIDER_MISSING	没有真实执行提供方，不能伪造观察结果
NOT_RUN	没有实际执行，不能写成 PASS
INFRA_ERROR	环境或设备故障，不是有效 RED，也不是业务 PASS
N/A	不适用，必须说明理由
GAPS_OPEN	存在未映射、未实现或未验证差距
FAILED	存在明确失败记录

CLI 常见退出码：0 表示当前命令完成，2 表示输入错误，3 表示确认或冲突，4 表示外部执行失败，5 表示部分完成或等待后续动作。预览退出 0 不等于业务完成。

提示词中的尖括号怎么替换

把：

```
目标目录：<项目目录>
```

替换成：

```
目标目录：/path/to/projects/task-helper
```

不要保留解释性文字，例如不要把 <模块甲> = <仓库 URL> 原样发给 Agent。没有偏好时可以写“由你先提出方案”，不要编造路径、哈希、设备或验证结果。

推荐的两段式口令

第一段只做预览：

```
先按对应工作流做只读检查和 dry-run。告诉我目标、范围、步骤、
文件变化、权限需求、产物和未验证边界。在我确认前不要正式写入、
安装、联网、执行目标、连接设备或发布。
```

第二段只执行已经确认的范围：

```
按刚才预览的范围继续，不要扩大目标、目录、工具、设备或发布范围。
完成后报告实际改动、验证结果、未执行项、冲突恢复状态，以及提交、
推送、归档和知识摄入状态。
```

第二部分：15 条主工作流保姆级教程

工作流 1：工程初始化

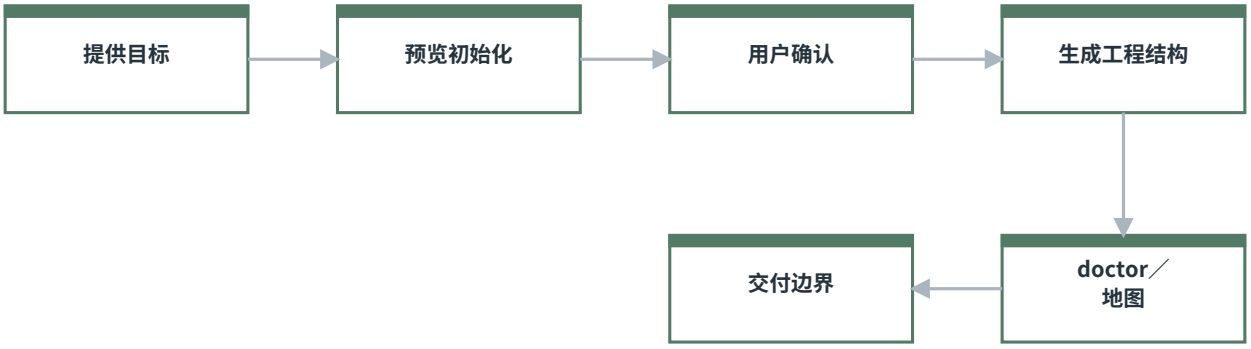


图 2 工程初始化：先预览，后生成

作用

把全新需求、已有工程或多个模块仓库，转成统一且可继续开发的工程结构。

总流程

```
启动输入 → intake dry-run → 按输入类型分流 → prepare / clone
→ 有源码则分批建图，无源码则写设计草案 → 生成 dossier 和模块资料
→ 可选索引、hooks、OpenSpec → accept-docs → doctor → handoff
```

路线 A：只有一句全新需求

```
需求 → 预览 → 建立公共入口和空知识库 → design-input.md
→ 项目资料与 handoff → NEEDS_DESIGN_APPROVAL
→ 用户确认设计 → OpenSpec 和开发计划
```

不要为了显得完整而创建拟议业务模块的空目录。设计未确认时可以交付草案，但不能声称业务初始化全部完成。

提示词：

```
请使用当前公共工程壳初始化一个全新项目。

BOOTSTRAP 位置：<路径或“当前仓库”>
目标目录：<项目目录>
项目名称：<名称>
项目目标：<要做什么>
目标用户：<使用者>
核心功能：<功能列表>
非目标：<不做什么；可写“由你梳理”>
技术约束：<平台、语言、部署；可写“由你提议”>

请完整读取 BOOTSTRAP 和项目规则，由你生成内部参数和哈希。
现在只做只读分析和初始化 dry-run，输出：
1. 推荐模块及职责；
2. 初始化后的目录树；
3. 设计草案大纲；
4. 后续 OpenSpec、TDD 和验证路线；
5. 需要我确认的设计决定；
6. 权限、文件变化和未验证边界。

不要正式写入、创建拟议模块空目录、git init、安装依赖、
运行业务代码、提交、推送或写入知识内容。
```

预览后：

```
按刚才预览的范围执行初始化。完成后运行 doctor，分别报告
initialization、documentation、coverage、handoff 和 runtime。
设计仍需确认时保留 NEEDS_DESIGN_APPROVAL，不要声称业务已完成。
```

路线 B：已有本地工程原地接入

```
读取现有规则 → 核对 Git 根和未提交修改 → 原地 prepare
→ 分批分析源码 → 关系地图 → 项目和模块资料 → doctor
```

提示词：

```
请把已有工程接入当前公共工程壳。

工程壳位置：<路径>
已有工程目录：<路径>

先只读预览：
1. 识别真实源码、模块边界、Git 根、HEAD 和未提交修改；
2. 读取现有人工 AGENTS 和模块局部规则；
3. 给出原地 prepare、模块登记、建图和资料生成计划；
4. 列出语义冲突、未覆盖范围和所需权限。

保留源码位置、人工文档和已有修改。不要移动源码、覆盖规则、
执行业务代码、自动安装依赖、提交或推送。
```

路线 C：多个命名模块仓库

启动单可以写成：

```
# 工程启动单
项目：设备助手
## 模块仓库
设备端 = https://example.com/team/device.git
服务端 = https://example.com/team/service.git
```

需要指定版本时：

```
## 来源版本
设备端 = branch:release/2.x
服务端 = tag:v1.0
```

URL 只是示例，必须换成自己的来源，不要把账号、密码或 Token 写进 URL。

提示词：

```
请初始化一个多模块工程。

目标目录：<路径>
模块清单：
<模块甲> = <仓库 URL>
<模块乙> = <仓库 URL>

先 dry-run，保留我给出的模块名称，分别规划到 modules/<名称>。
逐模块说明来源、目标路径、局部规则和后续建图方式。
不要自动启用 submodule、LFS、hooks 或凭据 helper。
部分失败时保留成功结果，不重新克隆全部或删除冲突目录。
```

初始化恢复和完成判断

部分完成时直接说：

```
继续上次初始化。先读取私有启动区、handoff 和当前 doctor，
重验当前权限与来源。已经成功且仍匹配的模块不要重复下载；
输入或来源变化时先报告冲突，不要换启动区强行覆盖。
```

只有 initialization=COMPLETE 才能称为完整初始化；它仍不证明业务运行或验收成功。

工作流 2：日常开发

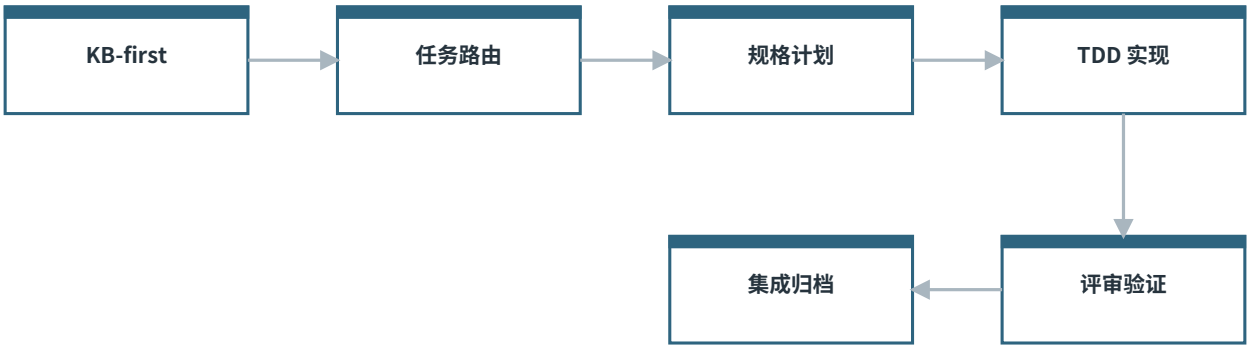


图 3 日常开发：规格与证据双闭环

作用

用统一入口处理查询、文档、缺陷、新行为、知识和设备／部署任务，让代码变化可追踪到需求、测试和证据。

入口顺序

```
AGENTS + handoff + 项目约束 + 模块局部规则
→ doctor → OpenSpec status/instructions/contextFiles
→ KB-first → 核对各仓库 HEAD 和已有修改 → 选择任务路径
```

路径 A：只读查询或审计

```
请只读审计当前工程的 <目标>。
重点范围：<目录、模块或工作流>。
分别给出已核实事实、推断、未验证事项和文件证据位置。
不要修改文件、更新地图、写入知识库或改变 OpenSpec 状态。
```

路径 B：纯文档修改

```
请修正 <文档范围> 中的 <问题>。
只修改指定内容，核对相对链接、内容归属、规则一致性和实际 diff。
不要制造代码 RED，不要顺带修改业务接口或验收含义。
```

路径 C：恢复既有规格的缺陷

```
请修复 <change/Requirement/Scenario> 对应的缺陷。
实际现象：<现象>；期望行为：<原规格要求>。
先复现和定位根因，形成有效 RED，再做最小修复和受影响回归。
不要修改既有验收含义，不要提交或推送。
```

路径 D：新行为、接口或跨模块设计

完整流程：

```
DISCOVERY → SPEC_VALIDATED → SPEC_APPROVED → PLAN_APPROVED
→ WORKSPACE_READY → APPLY_CONTEXT_LOADED
→ 每任务 RED → GREEN → REFACTOR → REGRESSION → TASK_VERIFIED
→ REVIEW → VERIFY → 用户选择提交/PR/保留 → 规格同步与归档
```

第一步提示词：

```
继续当前工程，开发：<功能目标>。
涉及模块：<模块或由你只读判断>。
必须满足的场景：<场景列表>。
兼容性和非目标：<约束>。

请从 handoff、doctor 和真实 OpenSpec 上下文继续。
先只做 DISCOVERY 和只读检查，给出拟议 Requirement、Scenario、
影响模块、接口变化、验证环境和实施计划。
在我批准规格和计划前不要写业务代码。
```

批准后提示词：

```
按已批准的 <change-id> 和计划实现。
每项任务执行有效 RED、GREEN、REFACTOR 和受影响回归，
通过后更新对应 OpenSpec task。
需求变化先更新规格；实现失败先定位根因；环境故障记 INFRA_ERROR。
不要提交、推送、归档或部署，先交付最终 diff 和验证证据。
```

路径 E：设备、部署或发布

先走适用的开发路径，再单独确认目标、当前状态、操作、验证和恢复方案。没有实际执行时保持 NOT_RUN，不要用开发测试替代设备或生产验收。

工作流 3：模块生命周期

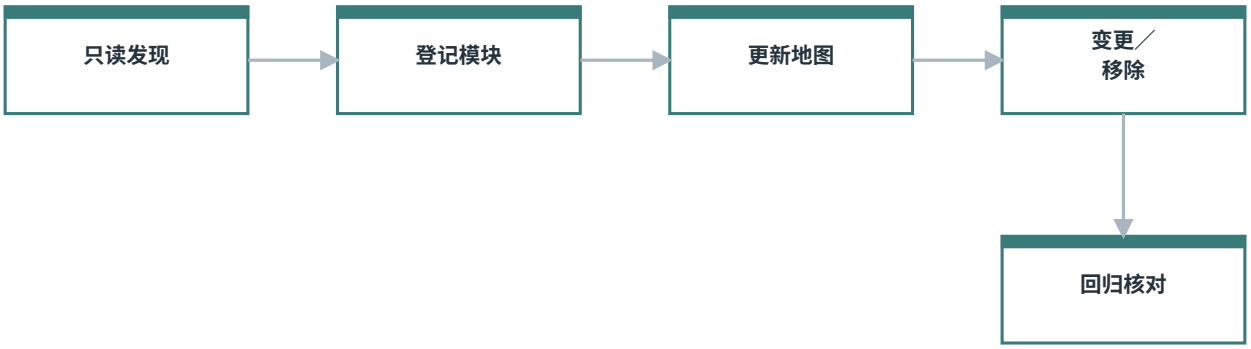


图 4 模块生命周期：地图随结构变化

作用

管理模块新增、接入、移动、删除、内部修改和对外接口变化，保持模块身份、资料、地图和验证一致。

流程

候选目录或 URL → 只读核对 Git 根和局部规则 → 确认 ID 和范围
→ clone 或 module add → 更新登记与资料 → 结构变化时建图
→ 模块测试 → 必要时集成测试 → doctor

提示词：

```
我要执行模块生命周期操作：
操作类型：<新增/接入/移动/删除/修改对外职责>
模块 ID：<module-id>
来源或当前路径：<URL 或路径>
目标路径：<路径>

先只读核对真实 Git 根、局部 AGENTS、依赖和接口责任。
说明是否改变模块边界或关系、是否需要发布地图、需要哪些局部和集成验证。
先给计划，不要直接 clone、移动或修改登记。
```

什么时候更新地图

- 新增、删除、移动模块：更新。
- 对外职责、模块间关系或整体拆分改变：更新。
- 只改模块内部实现：不重建地图，但源码指纹变化可能要核对项目资料。
- 只读任务和 CodeGraph sync：不等于发布地图。

多仓分别报告 HEAD、未提交差异和验证结果；多仓不是一个原子提交。

工作流 4：公共工程壳升级

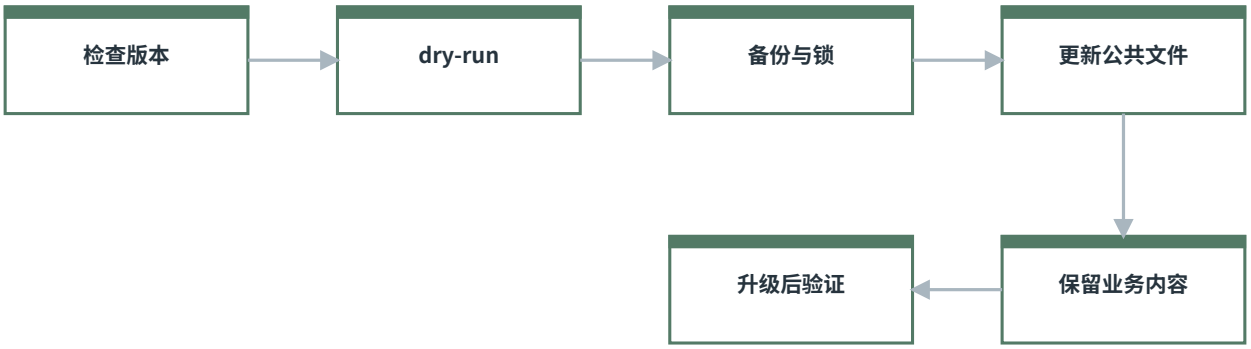


图 5 公共壳升级：公共文件与业务内容分离

作用

更新固定公共包和受管入口，同时保护业务源码、人工项目资料、模块规则、知识内容和活跃 change。

流程

```
确定目标版本 → 只读预览归属 → 独立升级 change
→ 更新 .agent-framework 和受管入口 → 只清理未修改的旧受管文件
→ 公共检查与业务回归 → doctor → 用户选择单独提交或保留
```

提示词：

```
请把当前项目的公共工程壳升级到 <版本或路径>。
先建立独立升级 change 并只读预览：
1. .agent-framework 和受管入口的变化；
2. 业务源码、docs/project 人工正文、模块规则、知识和活跃 change 的保护清单；
3. 人工修改或归属不明文件的冲突；
4. 公共检查、业务回归和回滚方案。

不要覆盖人工内容，不要归档业务 change，不要自动提交或推送。
```

工作流 5：知识检索与回流

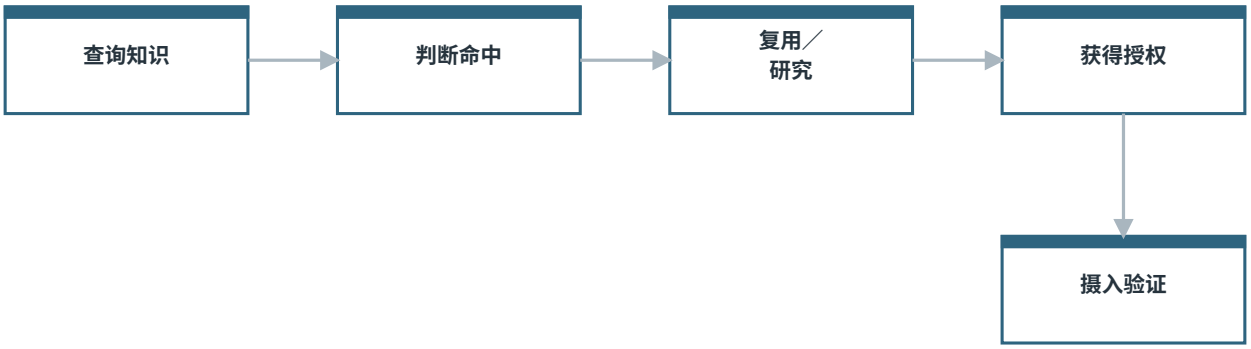


图 6 知识检索与回流：写入必须单独授权

作用

在任务开始时复用已有经验，在问题已验证解决后，把值得复用的内容写回正确知识库。

查询流程

确定 module-id → 找模块明确关联的 alias → 项目注册表解析 KB 根
→ 核对主题和 schema → 读 index → 只读查询相关页面

查询提示词：

请按当前项目 KB-first 规则查询：<要支持的决定或问题>。
涉及模块：<module-id>。
只查询已注册且明确关联的 KB，只保留能支持当前决定的内容，
区分来源事实与推断。不要建库、修改 alias、摄入或加载整库。

完整回流流程

问题已验证解决 → 列出内容、目标和验证范围 → 用户明确授权
→ 检索已有页面 → 准备来源与知识页 → 更新必要索引和日志
→ lint、链接和 diff 校验 → 本地提交与 push 分别决定

回流提示词：

本次问题已经验证解决：<问题摘要>。
涉及模块：<模块列表>。
拟保存内容：<根因、排查、修复、验证和限制>。
目标 KB：<alias 或路径>。

我授权按上述范围摄入。先检索已有页面，避免重复；
只保存脱敏、必要且可复核的内容，区分 EXTRACTED 和 INFERRED。
完成后检查 schema、链接、lint 和实际 diff，不要扩大目标或自动 push。

知识库其他操作怎么用

操作	实际作用	使用前提
init	创建空工程知识壳，不注册、不摄入	用户明确要求另建 KB；项目已有 knowledge/project/ 时不要重复建
register	注册项目内 alias	单独授权，拒绝冲突
query / query all	查询单库或全部项目 alias	只读，部分失败要如实报告
ingest / batch	准备不可变 raw 和待办，返回 PREPARED	授权写入；不等于完整摄入
digest	返回 PLAN_ONLY 和相关页，供 Agent 综合	后续综合和写入仍需授权与验证
crystallize	形成 INFERRED 综合计划	用户明确要求，不能把推理当发布事实
status / graph / lint	状态、图谱和结构检查	只读或核实模式；lint 不证明语义正确
delete	只给精确删除计划	先确认目标和引用，再可恢复删除并修复索引

另建知识库提示词：

```
我明确要求另建一个知识库：
用途和主题：<主题>
归属模块：<module-id>
目标目录：<路径>
alias：<名称>

请检查现有 KB 是否已适合、目录是否为空、alias 是否冲突。
先给建库和注册计划，不要覆盖已有内容，不要注册到全局。
```

删除知识提示词：

```
请为 <KB> 中 <raw 或知识记录> 制定删除计划。
列出精确目标、引用页面、索引和日志影响。
现在只给 PLAN_ONLY，不要按相似名称猜测或删除任何文件。
```

工作流 6：逆向观察

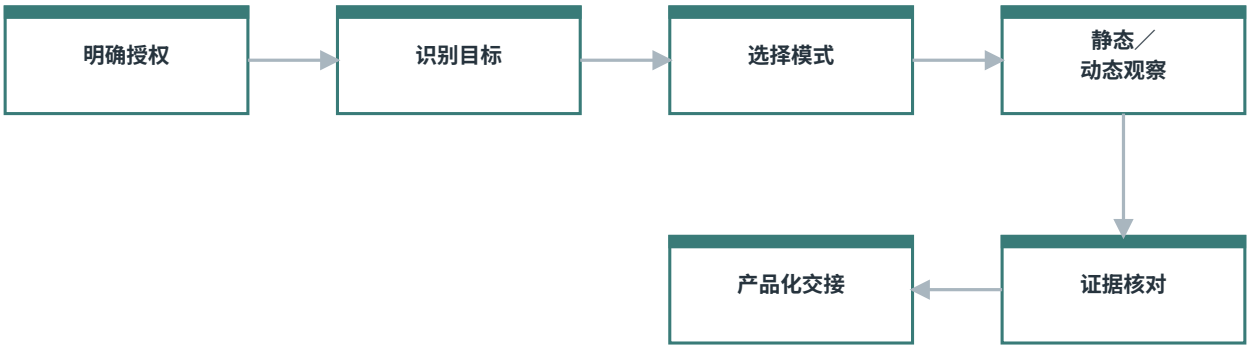


图 7 逆向观察：目标驱动工具选择

作用

在用户明确要求的范围内，选择合适的锁定工具和命名适配器，形成可复核的私有证据、行为契约和公开脱敏摘要，再回到普通设计与开发流程。

统一主流程

```
reverse-request/v1 或兼容 scope
→ 本地画像或 URL 授权快照 → 最小工具链计划 → dry-run
→ 用户批准计划哈希和具体权限 → 安装或健康复用
→ 命名适配器观察 → 私有证据和公开摘要 → 普通开发门禁
```

请求只描述目标、意图、验收、命名观察和证据，不提供任意 shell、argv、工具下载地址或任意版本。工具缺失且没有 reverse-install 时只报告缺口，不下载。

子模式 A：本地文件、目录、二进制或 WASM

```
本地内容 SHA-256 → 只读画像 → 最小工具计划 → 静态观察
→ 私有原文与行为契约 → docs/project/reverse/<id>.md
```

提示词：

```
我明确要求对本地目标进行逆向观察：
目标：<文件或目录>
希望理解：<接口、格式、算法或行为>
验收重点：<需要的结论和证据>

请使用自动 reverse-request 流程，由你生成请求和哈希。
先只读画像并 dry-run，给出最小工具、计划哈希、观察范围、
产物位置和限制。现在不要安装、联网或执行目标。
静态分析不得写成动态运行验证。
```

计划确认后：

```
批准刚才的逆向计划 <PLAN_SHA256>。
只执行计划中的最小工具安装和命名观察，不执行目标二进制，
不增加网络、浏览器或设备权限。分别报告工具链、观察、
runtime_validation 和证据状态。
```

子模式 B：网页或 Web 应用静态分析

```
dry-run → 单独授予 web-fetch → HTTPS 私有快照
→ HTML/JavaScript/WASM 发现 → 标准库静态资源分析
→ 私有证据与公开脱敏摘要
```

默认只允许明确 HTTPS origin，重定向和 DNS 结果重新检查，并限制资源数、总字节和超时。HTTP、localhost、私网和跨域必须在请求中明确限制，不能顺势扩大。

提示词：

```
我明确要求分析网页／Web 应用：
起始 URL：<完整 HTTPS URL>
允许的 origin：<origin 列表>
分析目标：<页面、JavaScript、WASM、接口或交互>
排除范围：<不需要的页面或数据>

先 dry-run，提出 origin、资源数、总字节和超时预算，
说明重定向、跨域、私网限制、最小工具、静态与动态能力边界。
当前不授权联网、登录态、浏览器或抓包，不绕过认证、反爬或混淆。
```

快照阶段：

```
我只授权刚才计划中的网页快照和静态资源分析。
遵守已确认的 origin 和预算，保存私有证据与公开脱敏摘要。
不使用登录态、真实浏览器或抓包，不扩大页面范围。
执行后报告实际获取内容、失败项和静态能力边界。
```

子模式 C：真实浏览器动态观察

需要 browser-observe 和真实 provider；默认 provider 未安装时返回 PROVIDER_MISSING，不能用静态结果冒充动态行为。既有登录态另需 web-auth，网络记录另需 network-capture。

提示词：

```
我补充授权以下动态观察：<DOM/console/截图/已注册观察>。
目标 origin：<已确认 origin>。
只使用命名观察和隔离会话，不使用既有登录态，不扩大资源范围。
如果 provider 不存在，返回 PROVIDER_MISSING，不要伪造结果。
原始 DOM、console、截图和网络记录只放私有证据区。
```

子模式 D：v1 兼容静态观察

旧五字段 scope 保留 id、target、sha256、question、action。action=static 调用本地 rea analyze；run 和 network 只记录未执行。旧集成可继续使用，新的目标驱动任务优先自动 request。

子模式 E：v2 多轮观察与行为契约

```
目标与基线 → 命名只读观察 → 私有原文 → 新观察追加
→ STATIC_SUPPORTED / DYNAMIC_OBSERVED / INFERRED 行为契约
```

每轮观察使用稳定独立 ID；同 ID 同请求幂等，同 ID 不同请求拒绝。行为契约只引用已成功且类型匹配的观察，推断不冒充实测。

提示词：

```
请为 <目标> 设计 v2 多轮观察。
目标是回答：<问题>；验收标准：<标准>。
列出每轮稳定观察 ID、命名操作、所需权限和能支持的行为结论。
先只读计划，不连接设备或网络。无 provider 时保留 PROVIDER_MISSING，
不要用推断补成动态事实。
```

子模式 F：设备试点、回滚和逐台复制

单设备试点：

```
命名设备与当前基线 → 独立备份及 SHA-256 → 核对计划操作和恢复操作
→ device-change → RECOVERY_READY → PILOT_APPLIED → PILOT_VERIFIED
→ ROLLBACK_READY
```

回滚是独立动作：

```
ROLLBACK_READY → 再次 device-change → ROLLED_BACK
→ runtime_validation 回到 NOT_RUN
```

逐台复制：

```
有效试点 + device-rollout → 核对每台兼容基线和独立备份
→ 逐台执行 → 每台独立结果 → 任一失败可形成 PARTIAL
```

试点提示词：

```
我要求评估 <命名设备> 的单设备试点。
目标变化：<变化>；验收：<验证>；恢复方式：<恢复>。
先只读核对设备身份、当前基线、备份文件及哈希、命名操作、
恢复操作和权限。默认操作锁没有真实修改命令时明确报告。
不要连接或修改设备，先给试点和回滚计划。
```

执行与回滚提示词：

```
只按刚才确认的 <计划/基线/备份> 执行单设备试点。
任一步失败立即停止，不自动重试，不扩展到其他设备。
分别报告备份、应用、验证和恢复就绪状态。
```

```
对 <试点 ID> 执行独立回滚。先核对 ROLLBACK_READY 和当前设备基线，
不要重新应用试点，不操作其他设备。完成后如实报告运行验证状态。
```

逆向产物和完成判断

- v1：.framework/local-state/rea/<id>.json 和私有原文。
- v2：.framework/local-state/reverse/v2/<id>/。
- 自动观察：.framework/local-state/reverse/auto/<id>/。
- 公开摘要：docs/project/reverse/<id>.md。
- evidence.raw=retain 保留私有原文和哈希；discard 只留哈希并标 RAW_DISCARDED。

doctor 只读检查目标、工具 receipt 和证据新鲜度，不执行观察、联网、连接设备或自动修复。

工作流 7：产品行为观察

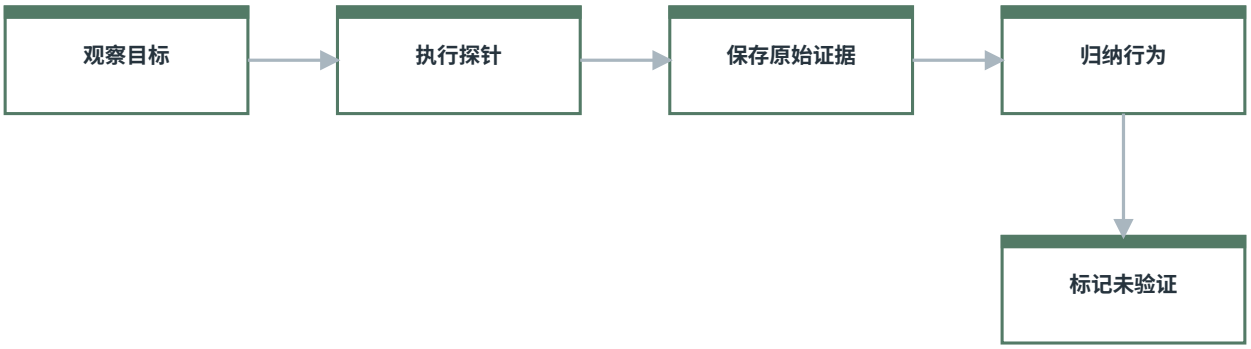


图 8 产品行为观察：事实、推断与未知分离

作用

把已经授权的原始观察整理成屏幕、流程、组件、实体等产品视图，并明确事实、动态观察、推断、未观察和排除范围。

流程

```
私有证据 → 当前 Agent 整理 product-recon-draft/v1
→ 稳定 S/F/C/E ID 和证据引用 → dry-run 校验
→ product-recon.json → 公开摘要只含统计和安全状态
```

提示词：

```
基于自动逆向 ID <ID> 的已授权证据整理产品行为观察。
希望重建的产品范围：<页面、流程、组件或实体>。

由你整理 product-recon-draft/v1：
1. 建立稳定 S/F/C/E ID；
2. 事实条目绑定当前有效且 provenance 相符的证据；
3. 区分 STATIC_SUPPORTED、DYNAMIC_OBSERVED、INFERRED；
4. 未观察保持未观察，明确排除范围；
5. 不编造截图、DOM、字段、接口或流程。

先 dry-run，不自动创建需求、批准设计或生成业务代码。
```

完整视图留在私有目录，公开摘要不复制名称、目的、步骤、字段、关系、排除原因或其他自由文本。runtime_validation 仍为 NOT_RUN，产品视图不是源码地图证据。

工作流 8：产品观察设计交接

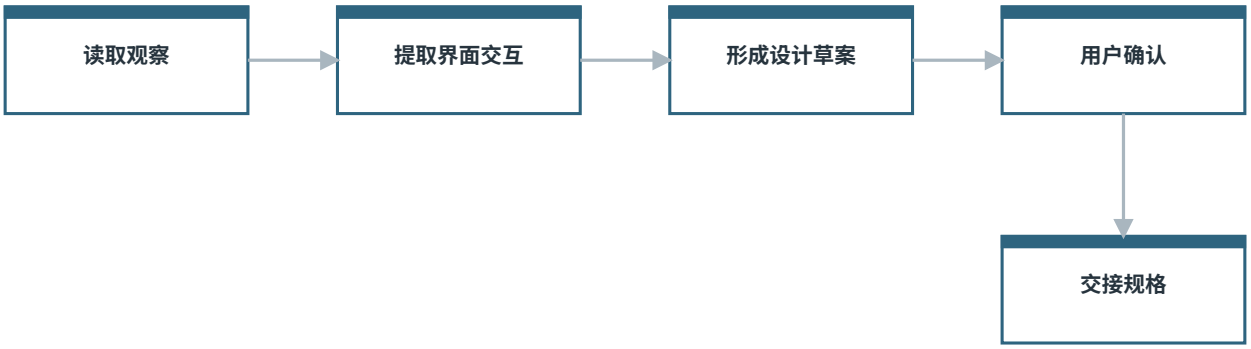


图 9 产品观察设计交接

作用

把产品观察转成公开、安全、可评审的新项目设计草案，再进入普通开发流程。

流程

```
选择自动逆向 ID → design-handoff dry-run
→ 核验请求、计划、契约、观察、目标和保留证据哈希
→ docs/project/design-input.md + 私有交接记录
→ NEEDS_DESIGN_APPROVAL → 用户确认 → DISCOVERY / OpenSpec
```

提示词：

```
请对自动逆向 ID <ID> 执行产品设计交接 dry-run。
核验请求、计划、行为契约、成功观察、目标与保留证据哈希。
检查公开草案只含白名单字段，并检测与人工草案的冲突。
不要重跑逆向、浏览器、网络或设备观察。
交接后保持 NEEDS_DESIGN_APPROVAL。
```

设计确认：

```
我确认当前 docs/project/design-input.md 的设计范围。
由你计算当前文件哈希并记录设计批准，然后进入 DISCOVERY 和 OpenSpec。
该批准不包含业务实现、提交、推送或发布授权。
```

交接不创建模块、不发布关系地图、不自动批准实现。人工草案或交接后编辑会冲突保留。

工作流 9：证据驱动功能差距追踪

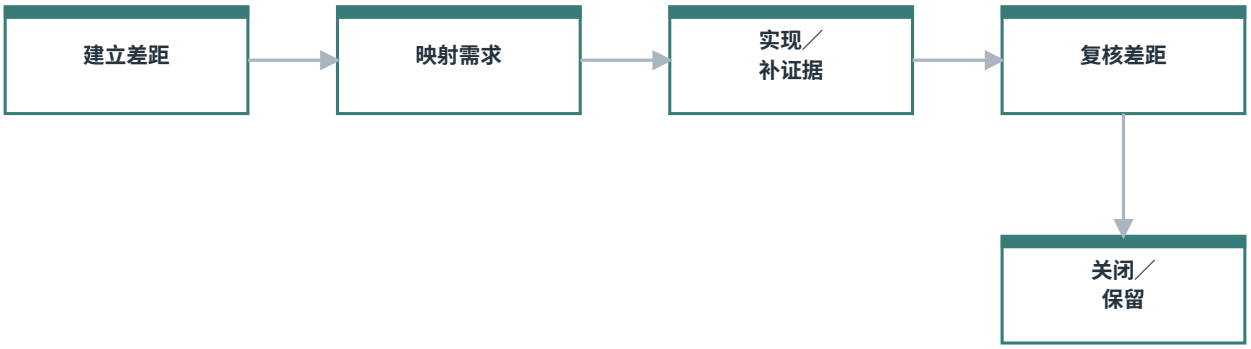


图 10 证据驱动功能差距追踪

作用

把产品目标、已批准场景、tasks 和验证记录放在同一追踪链，回答“哪些已验证、哪些未实现、哪些未验证、哪些失败”。

流程

```
产品观察 + 一个 change + specs + tasks + verification-evidence/v1
→ parity dry-run → 校验 ID、映射、路径和哈希
→ FUNCTION/PASS + 已完成 task + 有效 artifact 才支持 VERIFIED
→ S1–S4 BLOCK/REPORT 策略 → PASS/GAPS_OPEN/FAILED
→ 公开差距报告与私有回执
```

提示词：

```
请为 <change-id> 生成功能差距预检：
产品观察 ID：<ID>
验证证据目录：<docs/verification/<change>/>
S1–S4 策略：<BLOCK/REPORT 策略，或由你先列出建议>

检查产品 S/F/C/E 与 Requirement、Scenario、task、证据的映射，
以及 FUNCTION/PASS 的 task 和 artifact 绑定。
区分 FAIL、NOT_RUN、INFRA_ERROR、N/A 和视觉/可访问性/品牌诊断。
先 dry-run，不运行测试、不修改 task 或规格，列出全部差距。
```

产物：

- docs/verification/<change>-parity.md
- .framework/local-state/parity/<change>.json

工具只证明声明证据的结构和绑定可核验，不独立证明历史测试真实性。真实环境和评审仍由责任方完成。

工作流 10：公开市场研究交接

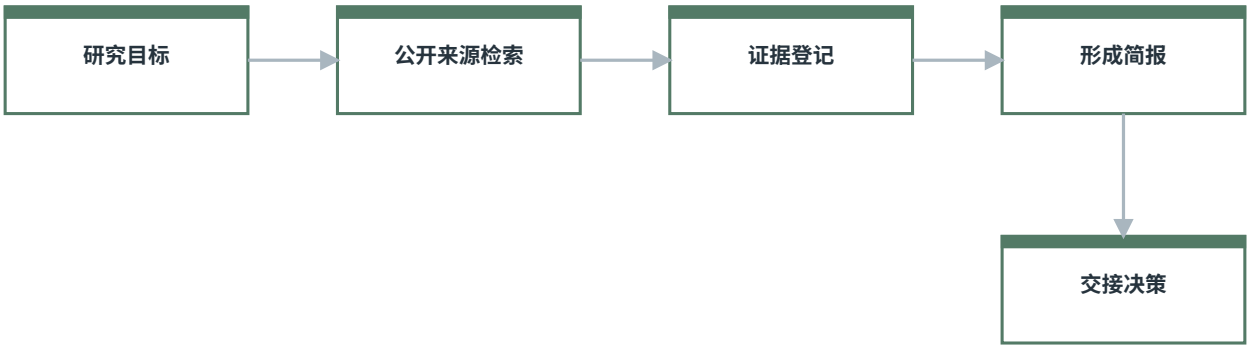


图 11 公开市场研究交接

作用

把竞品或市场资料整理成事实与推断分离的设计输入，避免研究结论直接变成已批准需求。

流程

```
用户授权公开研究 → 当前 Agent 获取资料
→ market-research-request/v1：来源、日期、内容哈希、事实、推断、局限
→ research dry-run → 公开报告和私有回执
→ NEEDS_DESIGN_APPROVAL → DISCOVERY
```

提示词：

```
请开展公开市场研究：
研究问题：<问题>
研究范围：<产品、地区、时间和来源>
公开来源要求：<要求或由你先列计划>

先给研究计划，不联网。后续授权后，由你获取资料并整理来源 URL、
访问日期、声明内容哈希、来源事实、Agent 推断、局限和设计草案。
最后 research dry-run，不把研究直接写成已批准需求或业务代码。
```

产物：

- docs/project/market-research/<id>.md
- .framework/local-state/market-research/<id>.json

工具本身不联网、不抓网页，也不验证来源仍在线或声明真实。固定保留 source_verification=NOT_RUN 和 execution=NOT_RUN。

工作流 11：发布候选检查

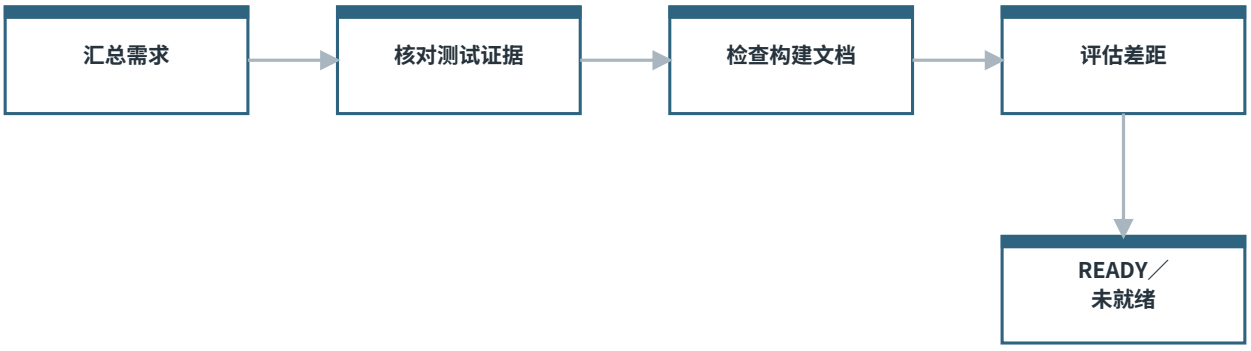


图 12 发布候选检查：证据不足则不能 READY

作用

把已经声明的构建和功能验证整理成候选门禁，判断明确允许公开的 dist/ 产物是否具备进入发布准备的条件。

流程

```
责任方已完成构建和验证 → release-candidate-request/v1
→ delivery-evidence/v1 和 dist/ 白名单 → candidate dry-run
→ READY / FAILED / GAPS_OPEN → 公开报告和私有回执
→ release=NOT_RUN
```

READY 至少需要一条 BUILD/PASS 和 FUNCTION/PASS；明确失败为 FAILED，缺通过记录或其他未执行项为 GAPS_OPEN。

提示词：

```
请检查当前产物能否形成发布候选：
候选 ID：<id>
版本：<版本>
源码版本：<commit 或版本>
公开产物：<dist/ 文件列表>
已有验证证据：<构建、功能和集成证据位置>

由你整理请求并先 dry-run，核验附件、结果、资产和哈希，
检查 BUILD/PASS 与 FUNCTION/PASS，区分 READY、FAILED、GAPS_OPEN。
不要运行构建或测试、复制资产、打 tag 或发布。
```

产物：

- docs/delivery/<id>-candidate.md
- .framework/local-state/delivery/<id>.json

候选只索引产物，不复制文件或生成安装包。证据真实性仍需责任方和适用评审确认。

工作流 12：发布渠道适配

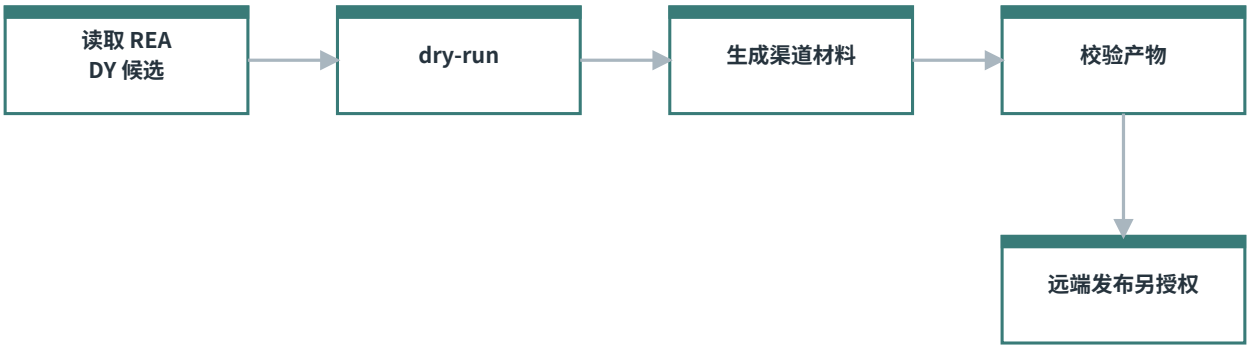


图 13 发布渠道适配：本地材料不等于远端发布

作用

将 READY 候选转换为具体发布渠道所需的本地产物，同时保持“候选就绪、渠道已适配、远端已发布”三种状态分离。

流程

```
READY 候选 → release-channel-request/v1
→ 绑定候选请求和报告哈希、公开目标、显式资产映射
→ channel dry-run 重核验候选、证据和当前资产
→ 复制所选资产原字节 + Release 说明 + 渠道清单
→ 公开报告和私有回执 → publish=NOT_RUN、execution=NOT_RUN
```

首版只支持 github-release，最多 100 项资产、总计 100 MiB。支持幂等、合法更新、人工冲突保护和目录／报告／回执分阶段失败恢复。

提示词：

```
请为 READY 候选准备 GitHub Release 本地渠道产物：
候选 ID 和请求：<ID、路径>
仓库：<owner/repo>
Tag：<tag>
标题：<公开标题>
draft：<true/false>
prerelease：<true/false>
资产映射：<源路径 → 公开文件名>

由你计算绑定哈希并先 channel dry-run。
重新核验候选、证据和当前资产，检查名称、数量和大小，
展示将生成的目录、说明、清单、报告和回执。
不要访问 GitHub API、打 tag、上传或创建真实 Release。
```

正式产物：

```
dist/delivery/<request-id>/
├── 显式选择的资产
├── release-notes.md
└── channel-manifest.json
docs/delivery/<request-id>-channel.md
.framework/local-state/delivery/<request-id>-channel.json
```

真实业务发布仍需后续独立规格与明确授权。

工作流 13：构建产物登记

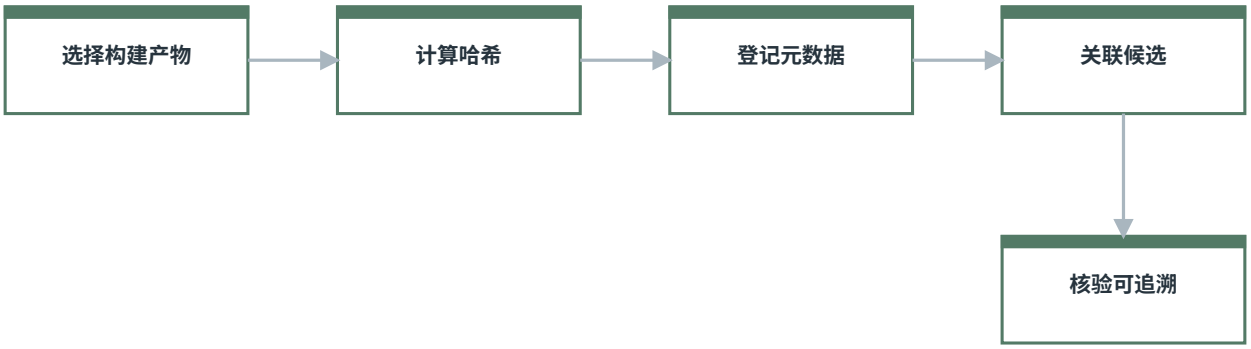


图 14 构建产物登记：从文件到可追溯证据

作用

为已经存在的 dist/ 文件生成哈希清单，便于后续核对产物是否变化。

流程

```
已有 dist/ → write + package → 扫描文件
→ 拒绝符号链接和私有/逆向路径 → 逐文件 SHA-256
→ package-manifest.json → runtime=NOT_RUN
```

提示词：

```
请登记当前项目已有的 dist/ 构建产物。
只扫描该目录，逐文件计算 SHA-256，拒绝符号链接和私有、逆向路径。
生成 package-manifest，保持 runtime=NOT_RUN。
不要执行构建、生成压缩包、复制文件或声称发布成功。
```

产物为 .framework/local-state/package-manifest.json。它不是构建、测试、候选或发布。

工作流 14：公共工程壳验证与 CI

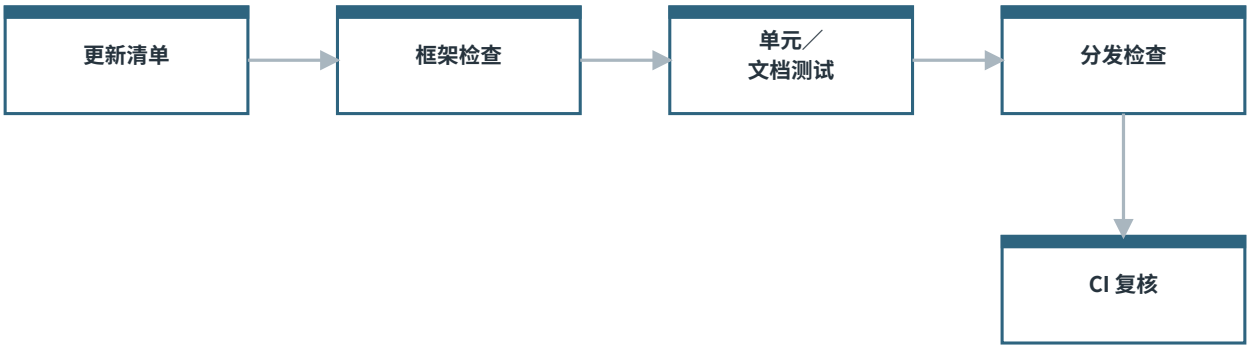


图 15 公共工程壳验证与 CI

作用

验证公共壳自身的分发完整性、文档链接、脚本语法和离线行为契约，并在 Ubuntu 与 macOS 上复验。

流程

```
公共壳变更 → 有效 RED/GREEN 与关联回归 → 更新分发清单
→ check-framework → 全量 unittest → OpenSpec 严格校验
→ git diff --check → 最终 diff 审查 → 提交推送
→ GitHub Actions 双平台检查
```

提示词：

```
请验证当前公共工程壳的 <change/工作树>：
1. 核对行为变更的 RED/GREEN 和关联回归；
2. 更新并审查 framework-manifest.json；
3. 运行 python3 tools/check-framework；
4. 运行全量 unittest；
5. 执行 OpenSpec 严格校验和 git diff --check；
6. 审查最终 diff、文件归属和未验证边界。
先不要提交、推送或发布；失败时定位根因并重跑适用检查。
```

CI 核验：

```
请检查提交 <commit SHA> 的 GitHub Actions 实际结果。
分别确认 Ubuntu 和 macOS 的状态、失败步骤和日志，
不要只根据本地测试推断 CI 成功。
```

CI 只证明覆盖范围，不证明真实宿主、私有认证、设备或生产部署。

工作流 15：公共工程壳版本发布

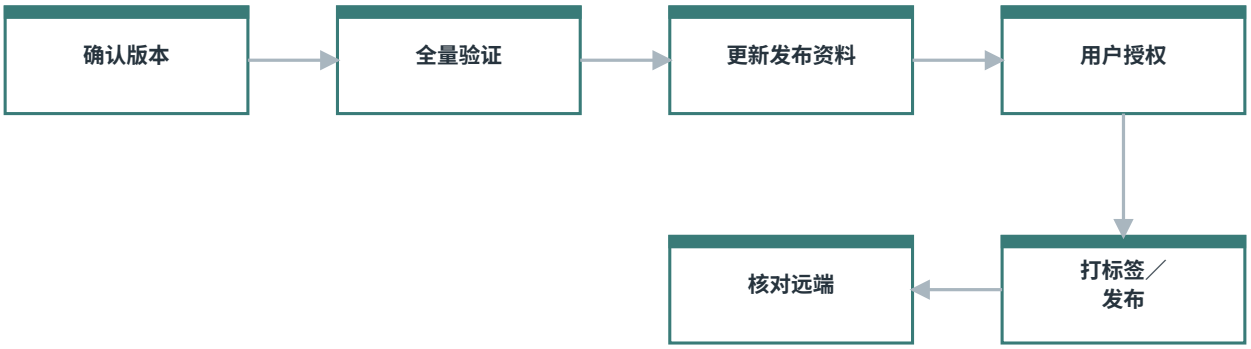


图 16 公共工程壳版本发布

作用

发布公共工程壳自身的新版本，不发布使用工程壳开发的业务项目。

流程

```
核对 Unreleased → 更新清单 → 公共检查和测试
→ 每周一定时或手动选择 patch/minor/major
→ 无说明则 SKIP；有说明则准备版本、changelog 和清单
→ 发布提交 → vX.Y.Z tag → 公共壳 GitHub Release → 发布后核对
```

定时为每周一 02:00 UTC，即北京时间 10:00。手动运行可选择 patch、minor 或 major。

发布前提示词：

```
请只读检查公共工程壳是否具备发布条件：
目标版本层级：<patch/minor/major>。
核对 Unreleased、分发清单、公共检查、测试、Git 状态、
兼容性、迁移和回滚边界。不要提交、打 tag 或触发发布。
```

明确发布提示词：

```
我确认发布公共工程壳自身的新版本。
版本层级：<patch/minor/major>；目标仓库：<owner/repo>。
按公共壳发布工作流执行，不发布任何业务项目。
完成后核对版本提交、tag、GitHub Release 和发布说明，
报告实际发布证据和未验证边界。
```

业务项目之后如需采用新版公共壳，另走工作流 4，不自动升级。

第三部分：组合场景，从零跑到终点

16. 场景一：只有一句想法，完成普通新项目

推荐路线

```
工作流 1A → 设计确认 → 工作流 2D → 适用验证
→ 可选工作流 9/11/12 → 用户决定提交、归档和知识回流
```

完整对话示例

你： 按“全新项目初始化”模板做 dry-run。

Agent 应返回： 模块建议、目录树、设计草案计划、权限和待确认项。

你：

```
按预览执行初始化，不创建未批准的业务模块，不运行项目代码。
```

你：

```
我确认当前设计。请建立第一批核心功能的 OpenSpec 规格和实施计划，
先不要写代码。
```

你：

```
按已批准计划实施。逐项完成 TDD 和回归，先不要提交。
```

最后检查： 设计批准、规格、任务、测试、最终 diff 和未执行项是否一致。

17. 场景二：已有工程接入并增加新功能

推荐路线

```
工作流 1B → 工作流 3（结构有变化时）→ 工作流 2D
→ 模块和集成验证 → 规格归档
```

完整对话示例

```
第一步：原地接入 <已有工程>，先只读预览，保留人工规则和源码位置。
第二步：按预览接入，完成地图、项目资料和 doctor。
第三步：继续开发 <功能>，先完成 DISCOVERY、SPEC 和 PLAN。
第四步：按批准计划 TDD，并运行模块与跨模块集成验证。
```

如果只有模块内部变化，不要为了“刷新”而重建关系地图；如果新增或移动模块，必须重新登记并在交付前发布地图。

18. 场景三：多模块仓库部分下载失败

正确做法

```
读取每模块结果 → 已成功且匹配的保留 → 失败模块分类
→ 缺权限则补权限，路径冲突则人工核对，来源不存在则修正来源
→ 从私有启动区恢复 → 不重新克隆全部 → 逐模块验证
```

提示词：

```
继续上次多模块初始化。
逐模块列出成功、失败、冲突和缺失原因。
已成功且来源、HEAD、路径仍匹配的模块不要重复下载；
失败或冲突模块只给最小修复建议，不删除目录、不覆盖用户文件。
```

19. 场景四：本地软件逆向后重新设计

推荐路线

```
工作流 6A → 工作流 7（可选）→ 工作流 8（可选）
→ 工作流 2 → 工作流 9 → 工作流 11/12
```

```
第一步：本地目标只读画像和工具链 dry-run。
第二步：批准指定计划，只安装必要工具并做命名静态观察。
第三步：根据有效证据整理产品视图，事实和推断分离。
第四步：设计交接，保持 NEEDS_DESIGN_APPROVAL。
第五步：确认设计后进入 OpenSpec 和 TDD。
第六步：用真实功能证据生成差距报告，不用逆向证据代替新实现测试。
```

20. 场景五：网页逆向重建产品

推荐路线

```
工作流 6B → 必要时 6C → 工作流 7 → 工作流 8
→ 工作流 2 → 工作流 9 → 工作流 11/12
```

每一步只扩大一次权限：

1. dry-run：不联网。
2. web-fetch：只获取确认 origin 的快照。
3. 静态分析：不启动浏览器。
4. browser-observe：只做命名动态观察。
5. web-auth：确实需要既有登录态时单独评估，不要默认开启。
6. network-capture：确实需要网络记录时单独授权。
7. 产品视图和设计交接：不重放目标。
8. 新实现：用新的测试和证据验收。

最关键的判断： 如果 provider 不存在，网页流程仍可做授权快照和静态资源分析，但不能宣称完成真实浏览器动态逆向。

21. 场景六：设备观察、试点和回滚

推荐路线

```
只读基线 → 命名观察 → 备份与恢复计划 → 单设备试点
→ 独立验证 → 独立回滚，或兼容设备逐台 rollout
```

必问清单

- 设备是否是请求中命名的那一台？
- 当前基线是否与计划一致？
- 备份是否存在、哈希是否匹配、恢复操作是否登记？
- 是否只有单台 device-change 授权？
- 失败是否停止而不是自动重试？
- 每台 rollout 是否有独立基线、备份和结果？
- 真实设备未执行时是否仍为 NOT_RUN？

22. 场景七：从构建产物到发布材料

推荐路线

```
业务构建与测试（责任方实际执行）
→ 工作流 13（可选哈希登记）
→ 工作流 11（候选门禁）
→ READY 后工作流 12（本地渠道适配）
→ 人工复核 → 后续独立发布规格和授权
```

不要把顺序倒过来：没有真实构建和验证证据，不能靠 candidate 或 channel 生成“发布成功”。

第四部分：常见问题与排查

23. 为什么初始化完成了，业务还不能说完成

因为初始化只证明工程接入、资料、覆盖和交接满足指定条件。业务完成还需要已批准 Scenario 对应的实现、测试、集成和适用环境证据。runtime 没有实际运行时仍是 NOT_RUN。

24. 为什么源码变了，地图还显示 CURRENT

模块内部源码变化不一定改变模块集合、路径或已引用关系，所以地图可以保持 CURRENT；但项目资料可能变 STALE，initialization 可能变 PARTIAL。需要核对资料原句，而不是机械重建地图。

25. 为什么不能直接安装所有逆向工具

工程壳采用最小能力闭包。只准备当前目标需要的工具，减少下载、攻击面和环境污染。锁文件没有校验值的工具不会自动下载，真实 provider 也不会因为工具名出现就凭空可用。

26. 为什么 dry-run 成功，正式执行仍可能失败

dry-run 证明当时输入和计划可解析，不保证后续权限、网络、设备、来源新鲜度或人工文件状态不变。正式执行会重新核对关键哈希、授权和冲突。

27. 为什么重复执行有时 UNCHANGED，有时 CONFLICT

- 输入和受管产物完全一致：UNCHANGED。
- 同一 ID 有合法变化且旧产物未被人工修改：UPDATED。
- 报告、回执、目录被人工修改，或出现额外／缺失文件：CONFLICT，保留现场。

28. 为什么测试 PASS 仍可能 GAPS_OPEN

可能是 MUST 目标未映射、Scenario 缺失、task 未完成、artifact 哈希不匹配，或者验证类别不是 FUNCTION。单个测试通过不能覆盖整个追踪链。

29. 为什么候选 READY 仍没有 Release

candidate 只评估候选证据和资产，channel 只生成本地渠道产物。二者都刻意不执行远端发布。只有公共壳自身的发布工作流接入真实 GitHub Release。

30. 为什么知识脚本 PREPARED 不是摄入完成

ingest 只准备 raw 和待办。完整摄入还需要 Agent 阅读实际来源、生成实质知识页、更新必要索引和日志、检查引用与 lint，并审查实际 diff。

31. 遇到冲突时最安全的通用口令

```
停止当前写入，保留现场。
列出冲突文件、当前哈希、预期来源、已完成的写入和恢复状态。
只给出最小处理方案，不要自动覆盖、删除、重建或扩大授权。
```

32. 遇到环境故障时的通用口令

```
把该现象记录为 INFRA_ERROR，不要当成有效 RED 或业务 PASS。
区分实现错误、依赖缺失、provider 缺失、网络失败和设备不可达，
给出恢复环境后需要重跑的最小验证集合。
```

第五部分：速查表

33. 我该从哪条工作流开始

我现在只有……	从这里开始
一句新项目想法	工作流 1A
一个已有本地工程	工作流 1B
多个模块 URL	工作流 1C
一个明确新功能	工作流 2D
一个已知缺陷	工作流 2C
一个文档问题	工作流 2B
一个只读审计问题	工作流 2A
一个新模块或模块移动	工作流 3
一个公共壳新版本	工作流 4
一个已解决问题要沉淀	工作流 5
一个本地逆向目标	工作流 6A
一个网页 URL	工作流 6B
一个产品观察 ID	工作流 7 或 8
一个已批准 change 和验证证据	工作流 9
一个市场研究问题	工作流 10
构建和功能验证证据	工作流 11
一个 READY 候选	工作流 12
一个已有 dist/	工作流 13
公共壳自身改动	工作流 14
公共壳 Unreleased 发布内容	工作流 15

34. 产物路径速查

产物	位置
项目事实和 handoff	docs/project/
公开逆向摘要	docs/project/reverse/<id>.md
自动逆向私有证据	.framework/local-state/reverse/auto/<id>/
产品完整观察	自动逆向私有目录中的 product-recon.json
公开设计草案	docs/project/design-input.md
功能差距报告	docs/verification/<change>-parity.md
市场研究报告	docs/project/market-research/<id>.md
构建产物哈希	.framework/local-state/package-manifest.json
发布候选报告	docs/delivery/<id>-candidate.md
渠道目录	dist/delivery/<request-id>/
渠道公开报告	docs/delivery/<request-id>-channel.md
渠道私有回执	.framework/local-state/delivery/<request-id>-channel.json
活跃规格变更	openspec/changes/
正式能力规格	openspec/specs/
公共壳分发清单	framework-manifest.json

35. 每次任务结束的通用检查提示词

```
请对本次任务做最终交付核对，不要扩大改动：
1. 实际目标和范围是否完成；
2. 修改了哪些文件，是否有越界或无关改动；
3. 每个适用 Requirement/Scenario 对应什么测试和证据；
4. 哪些检查实际执行并通过，哪些是 NOT_RUN、INFRA_ERROR 或 N/A；
5. 是否有冲突、部分失败或恢复动作；
6. doctor、地图和项目资料是否需要更新；
7. 是否只有自查，是否完成独立评审；
8. 最终工作树或提交标识是什么；
9. 提交、推送、集成、归档和知识摄入分别处于什么状态。
不要把文件存在、脚本启动或单个测试通过当成全部完成。
```

36. 新手最稳妥的十条规则

1. 先 dry-run，再正式写入。
2. 一次只授权当前阶段需要的能力。
3. 不把密码、Token、Cookie 或账号写进提示词、URL 和启动单。
4. 不让 Agent 为了“完成”而删除未批准需求或降低测试标准。
5. 不用静态地图、静态逆向或证据格式检查冒充真实运行。

6. 不用模块单测冒充跨模块集成。
7. 不让公共壳升级覆盖业务人工内容。
8. 不在冲突时自动覆盖、删除或重建。
9. 不把本地渠道适配当成真实发布。
10. 每次结束分别确认验证、提交、推送、归档和知识状态。

37. 现行规则深入阅读

- 初始化唯一入口：BOOTSTRAP
- 日常开发：daily-development
- 完整开发流程：development-workflow
- 目录管理：project-layout
- 模块生命周期：module-lifecycle
- 公共壳升级：framework-upgrade
- 测试与交付：testing-and-delivery
- 逆向工具与授权：reverse-tools
- 知识规则：knowledge-policy
- 公共壳验证：verification
- 公共壳发布：release

流程卡

项目	约定
输入	用户目标、目标目录或模块、当前 change、验证环境和具体授权
写入范围	只写当前工作流及已批准计划所属目录；dry-run 和只读审计不写入
授权	写入、下载、索引、逆向、网络、浏览器、设备、提交、推送、发布和知识分别核对
验证证据	实际命令输出、测试、最终 diff、工作树或提交、产物哈希和适用环境记录
完成状态	只声明当前工作流真实完成范围；初始化、业务运行、发布和知识摄入分别报告